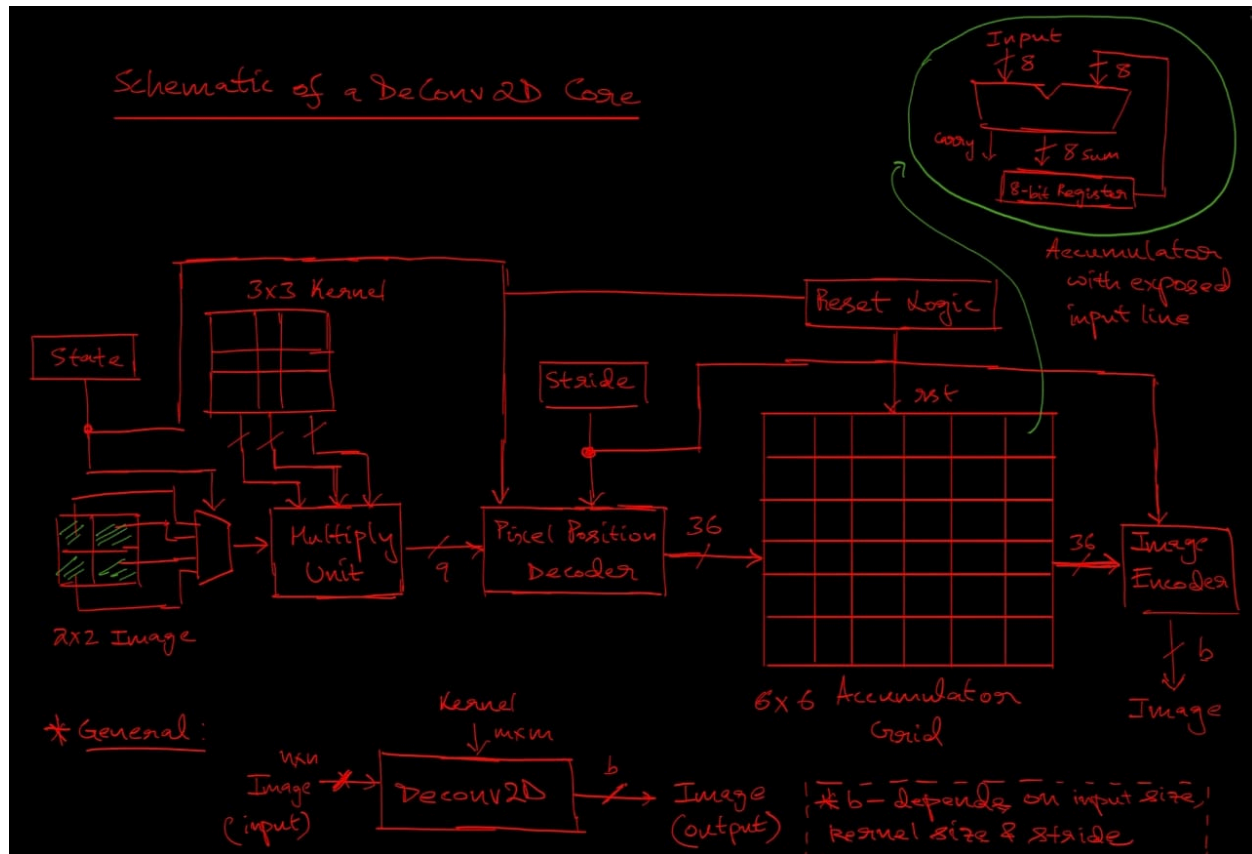


Task 1

Task 1 PS



- To design a Deconv2D kernel. Use the given schematic as reference.
- I have also drawn a multiplexer for the 2×2 input image which suggests that the 2D images must be flattened into 1D vector arrays, to feed to the multiplexer. The output image, should also be a flattened 1D image of the desired size. The image encoder and the pixel position decoder are probably the two most challenging aspects of the design. This must be designed in Verilog HDL on Vivado. When creating the project, you'll be given the option to choose your target board. Choose AMD Xilinx Kria KR260.
- Use behavioral code everywhere (the FSM using case blocks, multiplexers using if-else if-else blocks, and arithmetic operations using * and + operators

(ensuring DSP7 slices are used). First try working on the given fixed image and kernel sizes with variable size. Then parameterized the kernel and input image sizes. The goal is to reach the following utilization of the FPGA:

- Logic Cells used: ~40%
- DSP slices used: ~70%
- Assume every pixel is an 8-bit unsigned integer.
- 2×2 multiplied by 3×3 deconvolution circuit, Take input of 2×2 size (linearize or not) - integer 8 bit behavioural code, If I write all 36 possible output states, it will decide where the image should fit, according to the stride 9 input channel, 36 output channel, 2×2 input, and stride, Create 2×2 input Create 6×6 multiply and accumulator Create 9 to 36 decoder Generate blocks - can make multiple instances of the same module.

Important Universal Conventions and Notations while Coding

- All the lengths of arrays must be set according to variables M and N, not exact values 2 and 3; to ensure proper scaling later
K = 3 (Size of Kernel)
N = 2 (Size of input)
PIXEL_WIDTH = 8 (Width of pixel (8 bit integer))
ILEN = 8 ((Number of bits in the input of a particular module)
OLEN = 16 (Number of bits in the output of a particular module)
- When creating a module with inputs and outputs, keep track of the sizes of the wires. The truncation again to 8 bits will happen at the last output stage, but intermediate we will keep track of sizes (eg: 8bit * 8bit = 16 bit max)

Architecture Overview

Deconvolution Main Module: - **Deconv2D.v**

- This module will run the main FSM:

States:

IDLE - All modules off

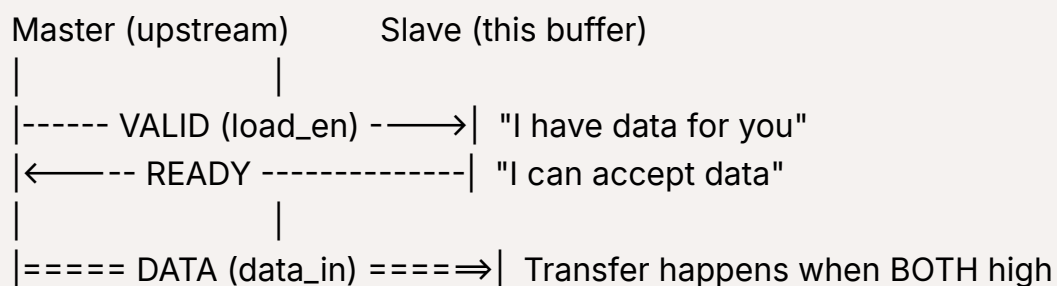
MULTIPLY - multiply modules active, compute the K^2 products

ACCUMULATE - Pixel Position Decoder will compute, then trigger accumulation of K^2 products into the output grid

Input Buffer module - `input_buffer.v`

Core Concept: AXI4-Stream-Style Handshaking

The buffer implements the ready/valid handshake protocol:



- **Handshaking** - Lossless data transfer with flow control.
- **Modularity** - Reuse for input, kernel, and later intermediate feature maps.
- AXI compliance.
- **Resource efficiency** - Automatic BRAM inference for large buffers.

BRAM-based FIFO/Queue than shift registers for GAN context:

- Multiply_unit needs all N^2 pixels simultaneously to compute K^2 products
- BRAM FIFO gives only 1 pixel/cycle → requires N^2 cycles to read buffer → throughput drops by N^2 factor
- Pure BRAM FIFO queues are ideal for inter-layer buffering and clock-domain crossing in multi-layer GAN pipelines, but not for the intra-layer sliding-window buffers that multiply unit needs.

Multiplexer module - `mux.v`

select bit to control input, and out_line is assigned to the value selected

```

module mux #(
    parameter N = 2,           // size of input (must match case entries)
    parameter WIDTH = 8       // width of each input
)(
    input wire [WIDTH-1:0] in_line [0:N*N-1],
    input wire [$clog2(N*N)-1:0] select,
    output reg [WIDTH-1:0] out_line
);
assign out_line = in_line[select];
endmodule

```

Multiply and Accumulate Unit: - **multiply_unit.v**

Parallel multiplier module

Compute K^2 products simultaneously for each input pixel.

Samples the kernel data from memory, and after that it gives a ready flag

```

module multiply_unit #(
    parameter K = 3,          // Kernel size
    parameter ILEN = 8,       // 8 bit integer input
    parameter OLEN = 16       // 8 bit integer product
) (
    // Inputs
    input wire [ILEN-1:0] input_pix, // single input pixel
    input wire [ILEN-1:0] kernel [0:K*K-1], // 1D kernel weights
    // Outputs
    output wire [OLEN-1:0] prod [0:K*K-1] // output 8 bit array of products
);

```

Accumulator modules: - **accumulator.v**

This module implements an accumulator for a grid of size $N*K$ by $N*K$.

It accumulates input values into a grid structure, where each cell in the grid corresponds to a position in the output image after deconvolution.

```

// Single accumulator cell
module accumulator_cell #(
    parameter ILEN = 16,
    parameter ACCUM_WIDTH = 20
)(
    input wire clk,
    input wire rst,
    input wire enable,
    input wire [ILEN-1:0] data_in,
    output wire [ACCUM_WIDTH-1:0] data_out
);

// corresponds to a position in the output image after deconvolution.
module accumulator #(
    parameter N = 2,
    parameter K = 3,
    parameter ILEN = 16, // width of each input segment
    parameter GRID_SIZE = N*K, // 6x6 grid
    parameter ACCUM_WIDTH = ILEN + $clog2(N*N) // width of running sum
) (
    // Inputs
    input wire clk,
    input wire rst,    // reset
    input wire enable, // accumulate when high
    input wire [ILEN-1:0] accum_in [0:N*K*N*K-1], // input to be accumulated
    // Outputs
    output wire [ACCUM_WIDTH-1:0] accum_grid [0:GRID_SIZE*GRID_SIZE-1]
);

```

Decoder: - **decoder.v**

this module will output the output grid by filling up the K^2 products in their positions, according to the stride and input index.

```

module decoder #(
    parameter N = 2,
    parameter K = 3,
    parameter ILEN = 8,
    parameter OLEN = 16,
    parameter stride = 2
)(
    input wire clk,
    input wire rst,
    input wire enable,
    input wire [$clog2(N*N)-1:0] state, // represents the current pixel index.
    input wire [OLEN-1:0] multiplied_image [0:K*K-1], // input image coming in from the multiplier.
    output reg [OLEN-1:0] decoded_image [0:N*K*N*K-1], // output image.
    output reg decoding_complete // indicates the completion of the decoding process.
);

```

Documentation

For an **N×N input image** and **K×K kernel** with **stride S**:

- **Number of input pixels** = N^2
- **Number of kernel weights** = K^2
- **Total multiplications per input pixel** = K^2 (broadcast one pixel to all kernel taps)
- **Output size** = $(N-1) \times S + K$ per dimension
- **Maximum operations** = $N^2 \times K^2$ multiply-adds

In each clock cycle during the state for computing:

1. FSM presents one of the N^2 input pixels via the mux
2. All K^2 multipliers compute in parallel (one clock of combinational delay or pipelined)

“Compute in parallel” means: Instantiate K^2 multiplier blocks in hardware - All see same pixel value, but different kernel[i]. In one clock period, all K^2 products are produced simultaneously.

In Verilog: You don't loop over time; you use a generate loop. At run time, all $\text{product}[i] = \text{pixel_in} * \text{kernel}[i]$; computations happen at once in hardware.

3. K^2 products are ready to be routed to the accumulator grid

MODULES REQUIRED:

1. Input Image Encoder:

a. Purpose: Take your 1D flattened input and present pixels with their 2D coordinates.

b. Input:

- `in_vector` in a RAM or register file (size = $2 \times 2 = 4$ entries for now)
- `start` from FSM

c. Internals:

- Compute 1D address: `in_addr = (ix-1)*IN_W + iy` ($\text{IN_W}=2$)
- Read `pixel = in_mem[in_addr]`

d. Outputs:

- `pixel_value` (8 bits).
- `ix`, `iy` to the position decoder.
- `valid_pixel` handshake to FSM/MAC.

2. Pixel Position Encoder

a. purpose:

- to give the passed multiplication matrix correct index in our new $W \times W$ matrix

b. Input:

- index of multiplexed K matrix

- index of image pixel which got multiplexed with kernel
- stride size
- new width of o/p feature map $W = N * K * N * K$

c. internals:

- compute new coordinates in strided output feature map
- $(i_k + s * i, j_k + s * j) \Rightarrow (i_k + s * i) * W + (j_k + s * j)$

d. outputs:

- output feature map after complete deconvolution in serial order to deconv 2d top module.

Issues Faced

- Faced issue with heavy computation for loop
- used fsm instead and performed decoding in K^2 cycles
- Initializing 6*6 matrix to zero as we get a thorough output of 4 6*6 matrix irrespective of strides
- Initially used an always block with an FSM in a generate block, which is incorrect, so we changed the code entirely.

Important extras

BRAM - instead of creating a shift reg buffer, use a queue:

Block RAM (BRAM) is hard silicon memory blocks embedded in Xilinx FPGAs, distinct from distributed RAM (made from LUTs):

How Vivado Infers BRAM

Inference means: Vivado's synthesis engine **recognizes a coding pattern** in your RTL and automatically maps it to a BRAM primitive instead of using LUTs. You don't instantiate BRAM primitives manually—you write clean HDL, and Vivado

decides.

Without dedicated module: Vivado may not infer BRAM (scattered logic)

With dedicated module: Clean memory access pattern → automatic BRAM inference → saves many flip-flops!

Conditions for BRAM Inference

Vivado will infer BRAM when **all** these conditions are met:

1. **Array declaration** with sufficient depth (typically ≥ 32 elements)
2. **Synchronous read** (read data registered on clock edge)
3. **Synchronous write** (write happens on clock edge)
4. **Single or dual-port access** (one or two read/write ports)
5. **Array is not partially accessed** (no bit-slicing that breaks BRAM structure)

Verifying BRAM Inference in Vivado

Step 1: Synthesis Report

After running synthesis, check the Utilization Report:

Location: <project>.runs/synth_1/<top>_utilization_synth.rpt

+-----+-----+-----+		
Site Type	Used	Avail
+-----+-----+-----+		
Block RAM Tile	2	144
RAMB36/FIFO	1	144
RAMB18	2	288
+-----+-----+-----+		

If BRAM count = 0 when you expected inference:

Array is too small (< 32 elements) → uses distributed RAM

Read is combinational (dout = ram[addr]; without clocking) → uses LUT-RAM

Array is partially accessed or has unsupported pattern

Step 2: Synthesis Log

Search synthesis log for:

Inferred 2 RAM(s) for "weight_mem"

RAM style: block (RAMB18E2)
or warnings:

[Synth 8-3936] Inferring RAM for variable 'kernel'
[Synth 8-7053] Using distributed RAM for 'kernel' (array too small)
Step 3: RTL Schematic
Open Synthesized Design → Schematic

Look for rectangular "RAM" blocks labeled RAMB18E2 or RAMB36E2

Click to see configuration (depth, width, mode)

Trace connections to confirm read/write ports

Timing errors - setup time and hold time:

AXI Architecture:

AXI (Advanced eXtensible Interface) is a high-performance, pipelined bus protocol that provides standardized, robust communication between modules in FPGAs and SoCs. It is a core part of the ARM AMBA specification and widely adopted in Xilinx platforms (like Kria KR260) and for interfacing IP blocks, accelerators, and memory controllers.

Signed and Unsigned bytes:

(make the modules such that it is easy to modify usage according to signed and unsigned)

Why Signed Is Necessary

- **Weights in GANs are bipolar:** Neural network weights (including GAN generator/discriminator) are trained around zero and take both positive and negative values. Using unsigned would require an offset bias, adding extra hardware and complicating dataflow.
- **Activation functions matter:** If you use ReLU in your discriminator, post-ReLU activations are non-negative and *could* be unsigned. However, Leaky ReLU or

Tanh (common in GANs) produce negative values, requiring signed representation.

- **MAC operations:** Multiply-accumulate results in convolutions can be negative (negative weight × positive activation), so accumulators must be signed regardless.

Code Review and Bugs (Vivado)

Vivado Warning: Parallel Synthesis Criteria Not Met (Performance Optimization)

`text[Synth 8-7080] Parallel synthesis criteria is not met`

What This Means - Vivado tried to **synthesize multiple modules in parallel** (using multiple CPU cores) to speed up compilation, but your design is **too small or too simple** for this optimization to help. This is **purely an optimization message**, not a functional problem.

Parallel synthesis requires:

- Multiple independent hierarchical modules
- Each module large enough (>500 cells) to justify parallelization overhead
- Design hierarchy preserved (not flattened)

Your `parallel_multiply` module is probably **too small** (9 DSP slices + minimal control logic). Vivado says: "This is so simple, I'll just synthesize it single-threaded." This warning is **completely harmless** for your deconv2D core at this stage.

When it matters:

- Large SoC designs with 50+ modules
- Synthesis taking >30 minutes
- You want to reduce compile time from hours to minutes

For your GAN accelerator:

- Synthesis probably takes 2-5 minutes → parallelization won't help
- Once you integrate `deconv2D` top module with multiple layers, Vivado might auto-enable it

Verilog Reference Guide

Verilog Cheatsheet (1)